

Universidad Nacional Autónoma de México

Facultad de Ingeniería

Sistemas Operativos



Grupo 1

Ing. Angel Isidro Mercado

Tarea 4. Sistemas de Archivos

320304435 - Cabrera Rojas Oscar

Ciudad Universitaria, CDMX a 02 de octubre del 2026

Índice

1. Definición y características de los archivos **1**
- 1.1. Concepto y naturaleza lógica del archivo . . . 1
- 1.2. Atributos y metadatos del archivo . . . 2
- 1.3. Estructuras del núcleo y gestión de archivos abiertos . . . 4
- 1.4. Operaciones del núcleo sobre archivos . . . 5
- 1.5. Organización interna y métodos de acceso . . . 7

2. Taxonomía y catálogo de formatos de archivo **8**
- 2.1. Mecanismos de identificación: extensiones, tipos MIME y números mágicos . 8
- 2.2. Catálogo técnico clasificado de cincuenta formatos de archivo . . . 9
  - 2.2.1. Documentos estructurados y ofimática . . . 9
  - 2.2.2. Código fuente, scripts y lenguajes de programación . . . 9
  - 2.2.3. Gráficos digitales y mapas de bits . . . 10
  - 2.2.4. Audio, vídeo y medios interactivos . . . 10
  - 2.2.5. Archivos comprimidos, empaquetados e imágenes de disco . . . 11
  - 2.2.6. Binarios ejecutables, bibliotecas y volcados de memoria . . . 11
  - 2.2.7. Bases de datos, configuración y serialización estructurada . . . 12

3. Arquitectura y componentes de los sistemas de archivos **12**
- 3.1. Definición formal y modelo arquitectónico por capas . . . 12
- 3.2. Capa de abstracción: el Sistema de Archivos Virtual (VFS) . . . 14
- 3.3. Estructuras en disco y en memoria principal . . . 14
- 3.4. Métodos de asignación de bloques en almacenamiento secundario . . . 15
- 3.5. Gestión del espacio libre en disco . . . 16

4. Análisis comparativo de sistemas de archivos **17**
- 4.1. Criterios de evaluación y taxonomía arquitectónica . . . 17
- 4.2. Tabla comparativa técnica de diez sistemas de archivos . . . 18
- 4.3. Análisis de paradigmas de consistencia y almacenamiento . . . 18

5. Sistemas de archivos en sistemas operativos contemporáneos **20**
- 5.1. GNU/Linux Mint 21.2 «Victoria» . . . 20
- 5.2. Ubuntu 23.04 «Lunar Lobster» . . . 20
- 5.3. Apple macOS 10.15 «Catalina» . . . 21
- 5.4. Microsoft Windows 10 . . . 21

6. Conclusiones personales **22**

Referencias **23**

1. Definición y características de los archivos

1.1. Concepto y naturaleza lógica del archivo

El subsistema de gestión de almacenamiento es uno de los componentes más visibles y fundamentales dentro de la arquitectura de cualquier sistema operativo moderno. Desde la perspectiva del usuario y de las aplicaciones en ejecución, la memoria de acceso aleatorio (RAM, *Random Access Memory*) resulta un medio volátil, de capacidad acotada y dependiente del suministro ininterrumpido de energía eléctrica, lo que impide la conservación indefinida del estado de los programas y sus datos. Para resolver esta limitación, el sistema operativo implementa una abstracción uniforme sobre los dispositivos físicos de almacenamiento secundario no volátil (discos magnéticos, unidades de estado sólido basadas en memorias flash NAND, medios ópticos o arreglos en red), ocultando la topología de sectores, bloques físicos, pistas y comandos de bajo nivel del controlador de hardware (Silberschatz et al., 2006, §10.1, p. 333).

Formalmente, un archivo es una colección con nombre de información relacionada, representada como una secuencia contigua en el espacio de direcciones lógicas y almacenada de forma persistente en memoria secundaria (Carretero Pérez et al., 2001, §8.2.1, p. 421). El archivo constituye la unidad lógica mínima de asignación y persistencia que el sistema operativo expone al usuario; ninguna aplicación puede almacenar datos en un medio persistente sin asociarlos a la entidad de un archivo reconocido por el sistema (Silberschatz et al., 2006, §10.1, p. 333). Esta entidad posee propiedades indispensables:

- **Persistencia temporal prolongada:** la información subsiste a la terminación del proceso que la generó, al cierre de la sesión del usuario e incluso a reinicios abruptos o fallos en el suministro eléctrico del equipo anfitrión (Stallings, 2018, §12.1, p. 551).
- **Identificación simbólica e independencia:** el archivo posee un nombre textual independiente del proceso creador, lo que permite su localización, referenciación y transferencia a través de diferentes entornos informáticos (Silberschatz et al., 2006, §10.1.1, p. 334).
- **Control de concurrencia y coutilización:** dispone de mecanismos de sincronización, cerrojos y derechos de acceso que permiten a múltiples procesos compartir datos de manera coordinada y segura (Deitel et al., 2004, §13.3, p. 587).

En el diseño de sistemas operativos coexisten dos paradigmas arquitectónicos para modelar el contenido interno de un archivo: el enfoque basado en registros estructurados y el enfoque orientado a flujo continuo de bytes. En sistemas operativos heredados de computadoras centrales (*mainframes*) y entornos de procesamiento transaccional como IBM z/OS o DEC VMS, los archivos se conciben bajo una jerarquía estricta compuesta por campos elementales que forman registros lógicos de longitud fija o variable (Deitel et al., 2004, §13.2, pp. 586–587). En dicho modelo, el propio núcleo del sistema operativo comprende la delimitación de cada registro, proveyendo primitivas de lectura orientadas a unidades de registro completas. No obstante, esta aproximación sobrecarga el núcleo con una proliferación excesiva de rutinas de soporte para cada tipo de dato, restringiendo la interoperabilidad cuando surgen aplicaciones con formatos no previstos por los diseñadores del sistema operativo (Silberschatz et al., 2006, §10.1, p. 341).

Por el contrario, el modelo introducido por UNIX y consolidado en los estándares de la interfaz de sistema operativo portátil (POSIX, *Portable Operating System Interface*), adoptado igualmente por Windows y los núcleos modernos de Linux y BSD, concibe el archivo como una secuencia lineal no estructurada de bytes de 8 bits (*stream of bytes*) (Carretero Pérez

et al., 2001, §8.2.1, pp. 421–422). Para el núcleo, el contenido de un archivo es semánticamente agnóstico y carece de estructura predeterminada; cada byte es accesible mediante su desplazamiento numérico relativo al inicio. La responsabilidad de interpretar, segmentar y validar el flujo binario recae exclusivamente en las bibliotecas de tiempo de ejecución y en la lógica de las aplicaciones de usuario (Silberschatz et al., 2006, §10.1, p. 341). Esta ortogonalidad provee la máxima flexibilidad y simplifica el diseño del núcleo, permitiendo tratar de forma análoga archivos ordinarios, tuberías de comunicación y descriptores de dispositivos físicos bajo la premisa universal de que “en UNIX, todo es un archivo”.

A nivel físico, el hardware de almacenamiento opera invariablemente transfiriendo bloques de datos de tamaño homogéneo (comúnmente sectores lógicos de 512 bytes o bloques de 4096 bytes en formatos avanzados). El subsistema de archivos se encarga de empaquetar de forma transparente los bytes del flujo lógico dentro de los bloques físicos asignados. Cuando la longitud total de los datos no coincide con un múltiplo exacto del tamaño de bloque, la porción residual del último bloque queda sin utilizarse, fenómeno denominado fragmentación interna (Silberschatz et al., 2006, §10.1.5, p. 342).

1.2. Atributos y metadatos del archivo

Un archivo no se limita a su cuerpo de datos; el sistema operativo asocia y preserva de manera inseparable un conjunto de metadatos o atributos de control que describen su estado, permisos, procedencia y ubicación física en el medio (Stallings, 2018, §12.1, p. 552). Estos metadatos permiten a los subsistemas de seguridad, planificación y memoria administrar el archivo con eficiencia e integridad. Los atributos universales presentes en los sistemas contemporáneos son:

1. **Nombre simbólico:** Cadena de caracteres alfanuméricos destinada a la interacción con el usuario y la jerarquía de directorios. En sistemas con distinción de mayúsculas y minúsculas (*case-sensitive*) como Linux, `informe.txt` y `Informe.txt` constituyen archivos distintos, mientras que en sistemas insensibles a las mayúsculas como NTFS en Windows son equivalentes.
2. **Identificador unívoco:** Valor numérico binario único que identifica inequívocamente al archivo dentro de la estructura física del sistema de archivos (Silberschatz et al., 2006, §10.1.1, p. 334). En sistemas POSIX corresponde al número de nodo de índice (*inode* o inodo), mientras que en NTFS de Windows se materializa en el índice del registro de la tabla maestra de archivos (MFT, *Master File Table*) (Carretero Pérez et al., 2001, §8.2.1, p. 422). Las estructuras internas del núcleo operan siempre mediante este identificador numérico, independientemente del nombre simbólico o las rutas de acceso.
3. **Tipo de archivo y formato:** Especifica la naturaleza intrínseca del archivo. En entornos de escritorio se apoya en extensiones de archivo o identificadores de tipo MIME (*Multipurpose Internet Mail Extensions*). No obstante, en el nivel del núcleo se recurre al denominado número mágico (*magic number*), una secuencia fija de bytes grabada en los primeros sectores de la cabecera del archivo que valida inequívocamente su formato (por ejemplo, los bytes `0x7F 0x45 0x4C 0x46` para binarios ELF en Linux o `0x25 0x50 0x44 0x46` para documentos PDF) (Carretero Pérez et al., 2001, §8.2.1, p. 422).
4. **Ubicación y mapa de asignación:** Punteros y estructuras de direccionamiento que relacionan el espacio lógico del archivo con los identificadores de bloques lógicos

(LBA, *Logical Block Addressing*) o agrupaciones de sectores (*clusters*) en el dispositivo físico de almacenamiento (Carretero Pérez et al., 2001, §8.2.1, p. 422).

5. **Tamaño y consumo de almacenamiento:** Describe tanto la longitud lógica actual del archivo en bytes como la cantidad real de bloques físicos reservados en disco. En archivos dispersos (*sparse files*), el tamaño lógico puede ser de varios terabytes mientras que el espacio físico consumido se reduce a los pocos bloques que contienen datos distintos de cero.
6. **Control de acceso y protección:** Información de privilegios que dictamina qué entidades pueden ejecutar operaciones de lectura, escritura o ejecución. En entornos POSIX se implementa mediante la máscara clásica de nueve bits (lectura, escritura y ejecución para usuario propietario, grupo y otros), complementada por listas de control de acceso (ACL, *Access Control Lists*) que permiten asignar derechos granulares a usuarios o grupos individuales (Stallings, 2018, §12.1, pp. 552–553).
7. **Marcas temporales (*timestamps*):** Registros cronológicos de auditoría que documentan eventos críticos del ciclo de vida del archivo. En sistemas tipo UNIX y Linux se distinguen tres marcas elementales:
   - `atime` (*access time*): marca temporal del último acceso de lectura al contenido del archivo.
   - `mtime` (*modification time*): marca temporal de la última escritura o modificación del contenido de datos.
   - `ctime` (*change time*): marca temporal del último cambio en los metadatos o atributos del inodo (permisos, propietario, enlaces duros).
   - En sistemas modernos y en NTFS se incorpora además `crtime` / `btime` (*creation/birth time*), registrando el instante exacto en que fue creado el archivo (Carretero Pérez et al., 2001, §8.2.1, p. 423).

La Tabla 1 sintetiza los atributos analizados, contrastando su nomenclatura conceptual con su implementación real en sistemas tipo UNIX y en entornos Windows.

Tabla 1: Atributos y metadatos fundamentales de los archivos en sistemas operativos contemporáneos.

Atributo	Implementación en UNIX / Linux (Inodo / POSIX)	Implementación en Windows (MFT / NTFS)
Identificador	Número de inodo entero (`ino_t`), único por partición lógica montada en el árbol.	Identificador de referencia de 64 bits compuesto por el índice en la MFT y un número de secuencia.
Tipo	Campo `st_mode` codificado en bits (archivo regular, directorio, enlace simbólico, socket, tubería, dispositivo).	Atributos de registro en MFT y asociación mediante la extensión del nombre en el Registro de Windows.
Seguridad	Identificador de usuario (`uid`), de grupo (`gid`), bits `rwxrwxrwx`, SUID/SGID y atributos POSIX ACL.	Descriptor de seguridad (*Security Descriptor*) que integra el SID del dueño y una lista ACL discrecional (DACL).
Mapa de bloques	Árbol de punteros directos, indirectos simples, dobles y triples (o árbol de extensiones *extents* en ext4).	Lista de fragmentos de ejecución (*runlists*) que especifican pares de desplazamiento virtual (VCN) y lógico (LCN).
Temporización	Estructuras `timespec` para marcas `atime`, `mtime` y `ctime` con resolución de nanosegundos.	Estructuras `FILETIME` de 64 bits con precisión de 100 nanosegundos para creación, modificación, acceso y cambio.

1.3. Estructuras del núcleo y gestión de archivos abiertos

Para evitar lecturas repetitivas y costosas a las estructuras de almacenamiento secundario cada vez que un proceso requiere transferir datos, el sistema operativo implementa en memoria principal una jerarquía de estructuras de control que optimizan la gestión de los archivos abiertos (Silberschatz et al., 2006, §11.2, p. 372).

En disco, toda la información de control y mapa de un archivo se encuentra condensada en el Bloque de Control de Archivo (FCB, *File Control Block*) (Silberschatz et al., 2006, §11.2, p. 373). Al emitirse una solicitud de apertura mediante la llamada al sistema `open()`, el subsistema de archivos explora la estructura de directorios, valida los permisos de acceso y traslada una copia del FCB a las estructuras dinámicas del núcleo en memoria RAM. Esta coordinación se articula a través de tres niveles de abstracción:

1. **Tabla de descriptores de archivo por proceso:** Cada entrada en la tabla de procesos del núcleo (el Bloque de Control del Proceso o PCB) mantiene un arreglo de descriptores de archivo abiertos. El índice de este arreglo es el descriptor de archivo (*file descriptor* en POSIX, un número entero no negativo; o *file handle* en Windows). Esta tabla local almacena el modo de acceso asignado al descriptor y un puntero a una entrada específica dentro de la tabla global de archivos abiertos del sistema (Silberschatz et al., 2006, §11.2, p. 373).
2. **Tabla global de archivos abiertos del sistema (*system-wide open-file table*):** Alberga una entrada por cada archivo en uso en el sistema. Contiene el puntero de desplazamiento actual (*file offset*) compartido entre los descriptores vinculados a dicha entrada (por ejemplo, tras una llamada `fork()`), el modo de acceso efectivo (`O_RDONLY`, `O_WRONLY`, `O_RDWR`), el contador de referencias de aperturas activas (*open count*) y un puntero hacia la estructura de nodo virtual (*vnode*) o inodo en memoria activa (Silberschatz et al., 2006, §10.1.2, p. 336).
3. **Tabla de inodos en memoria activa (*vnode / inode table*):** Mantiene la copia sincronizada del FCB o inodo físico del disco, complementada con información puramente dinámica del núcleo: cerrojos de exclusión mutua, semáforos de sincronización, lista de páginas sucias en la memoria caché del sistema (*page cache*) y contador global de enlaces duros (Carretero Pérez et al., 2001, §8.6.2, p. 473).

La Img. 1 ilustra el flujo de punteros y estructuras de datos involucradas en la apertura y lectura coordinada de un archivo por parte de procesos independientes y procesos emparentados en el sistema operativo.

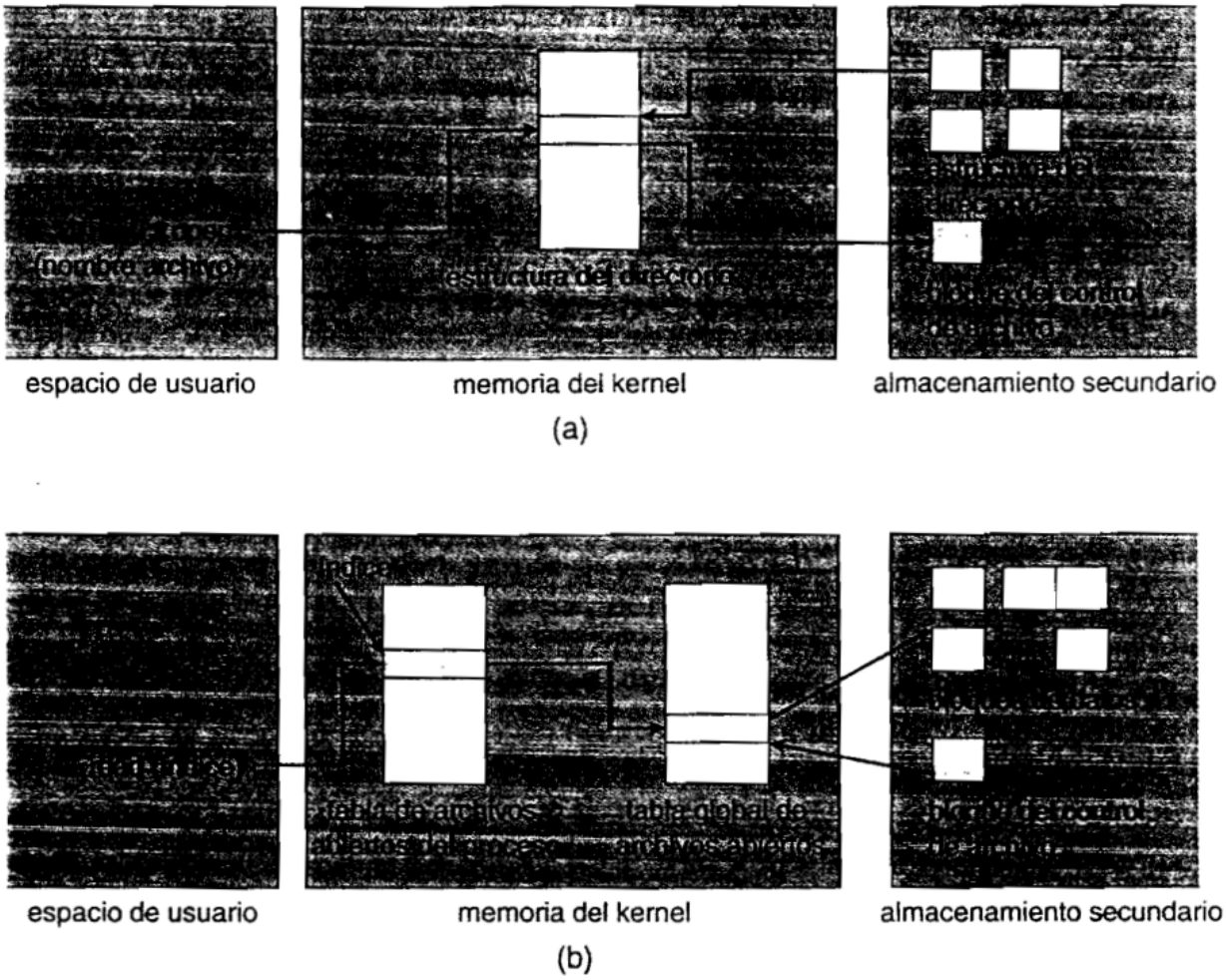


Imagen 1: Estructuras de control en memoria del sistema operativo para la gestión coordinada de archivos abiertos (Silberschatz et al., 2006, §11.2, p. 373).

Cuando un proceso invoca la primitiva `close()`, el núcleo elimina la entrada correspondiente en la tabla del proceso y decrementa en una unidad el contador de aperturas en la tabla global. Si dos procesos independientes abrieron el mismo archivo por separado, cada uno posee su propio puntero de desplazamiento en la tabla global, evitando que la lectura efectuada por un proceso altere la posición de lectura del otro. Únicamente cuando el contador de aperturas del archivo desciende a cero, el núcleo descarga las páginas modificadas a disco (*flush*), libera la entrada global y retira el inodo de la memoria activa si no existen otras referencias en caché (Silberschatz et al., 2006, §10.1.2, pp. 336–337).

1.4. Operaciones del núcleo sobre archivos

Un archivo constituye formalmente un tipo abstracto de datos, caracterizado tanto por su estado y atributos como por el conjunto de operaciones primitivas que el sistema operativo pone a disposición de las aplicaciones mediante llamadas al sistema (*system calls*) (Silberschatz et al., 2006, §10.1.2, p. 335). Las operaciones medulares implementadas en los núcleos contemporáneos son:

- **Creación (*create*):** Localiza espacio libre en la estructura del sistema de archivos para alojar el nuevo FCB o inodo, inicializa sus atributos (propietario, permisos, marcas de

tiempo en cero) y registra una nueva entrada en el directorio padre que vincula el nombre simbólico con el identificador numérico asignado (Silberschatz et al., 2006, §10.1.2, p. 335).

- **Apertura (*open*):** Verifica los derechos de acceso del proceso solicitante contra la ACL o los permisos del inodo, carga el FCB a memoria activa si no se hallaba previamente en uso, inicializa una entrada en la tabla global del sistema y asigna un descriptor no negativo en la tabla local del proceso, retornando dicho descriptor al espacio de usuario (Silberschatz et al., 2006, §11.2, p. 373).
- **Lectura (*read*):** A partir del descriptor proporcionado, consulta el puntero de desplazamiento actual, transfiere el número de bytes solicitado desde la caché de páginas del núcleo hacia el búfer de memoria del espacio de usuario y avanza el puntero de desplazamiento en una cantidad de bytes equivalente a los leídos (Silberschatz et al., 2006, §10.1.2, p. 335).
- **Escritura (*write*):** Transfiere bytes desde el búfer de la aplicación hacia las páginas de caché del núcleo, asignando nuevos bloques físicos en disco si la operación supera el tamaño actual del archivo, y avanza el puntero de desplazamiento. En sistemas con escritura pospuesta (*write-behind*), los datos se persisten en disco de forma asíncrona mediante demonios del núcleo (como `pdflush` o `kworker` en Linux), salvo que se invoque explícitamente una llamada de sincronización obligatoria como `fsync()` o `fdatasync()` (Carretero Pérez et al., 2001, §8.4.2, p. 441).
- **Reposicionamiento (*seek*):** Modifica la posición actual del puntero de desplazamiento relativo al inicio (`SEEK_SET`), a la posición actual (`SEEK_CUR`) o al final del archivo (`SEEK_END`), sin llevar a cabo ninguna transferencia física de entrada/salida sobre el dispositivo (Silberschatz et al., 2006, §10.1.2, pp. 335–336).
- **Truncado (*truncate*):** Reduce la longitud lógica de un archivo a un número determinado de bytes (frecuentemente cero) y devuelve al mapa de bloques libres el espacio sobrante en disco, preservando inalterados todos los metadatos, propietario y atributos de seguridad del archivo (Silberschatz et al., 2006, §10.1.2, p. 335).
- **Cierre y desvinculación (*close* y *unlink*):** `close()` disocia el descriptor del proceso liberando la entrada local y decrementando el contador global. Por su parte, `unlink()` o `DeleteFile()` suprime el enlace en el directorio; si el archivo no posee otros enlaces duros y su contador de aperturas es cero, el núcleo procede a liberar definitivamente todos los bloques de datos y el FCB/inodo en el medio persistente (Carretero Pérez et al., 2001, §8.4.2, p. 442).

La Tabla 2 compara directamente las llamadas al sistema estandarizadas en la norma POSIX de IEEE/The Open Group frente a las interfaces de programación nativas de Win32 provistas por el subsistema del núcleo de Windows NT.

Tabla 2: Comparativa de interfaces de llamadas al sistema para operaciones sobre archivos: POSIX frente a Win32.

Operación	Llamada al sistema POSIX (C / Linux / BSD)	Función de la API Win32 (Windows NT)
Apertura y creación	`open(path, flags, mode)`	`CreateFile(name, access, share, sec, disp, flags, temp)`
Lectura	`read(fd, buf, count)`	`ReadFile(file, buf, count, bytesRead, overlapped)`
Escritura	`write(fd, buf, count)`	`WriteFile(file, buf, count, bytesWritten, overlapped)`
Reposicionamiento	`lseek(fd, offset, whence)`	`SetFilePointer(file, distLow, distHigh, method)`
Truncado	`truncate(path, len)` `ftruncate(fd, len)`	o `SetEndOfFile(file)` tras posicionar el puntero del archivo.
Cierre de descriptor	`close(fd)`	`CloseHandle(handle)`
Eliminación	`unlink(path)`	`DeleteFile(name)`

1.5. Organización interna y métodos de acceso

La manera en que un archivo es leído y procesado por el software determina el método de acceso soportado por el sistema de archivos. Dependiendo de los requerimientos de latencia, concurrencia y ordenamiento de la aplicación, se distinguen tres métodos principales (Silberschatz et al., 2006, §10.2, pp. 337–340):

1. **Acceso secuencial:** Es el método más simple y extendido. La información contenida en el archivo se procesa en estricto orden lineal, byte por byte o registro por registro, desde el principio hasta el final (Stallings, 2018, §12.2, p. 556). Las operaciones de lectura consumen automáticamente el siguiente segmento de datos (`read_next`) y avanzan el puntero de archivo de manera implícita. Las operaciones de escritura añaden información de forma continua al final del archivo (`append`). Este esquema emula fielmente el comportamiento de dispositivos electromecánicos lineales como las cintas magnéticas y resulta óptimo para compiladores, analizadores de texto, volcado de registros de eventos (*logs*) y reproductores continuos de audio y vídeo (Silberschatz et al., 2006, §10.2.1, p. 337).
2. **Acceso directo o aleatorio (*direct / random access*):** En este modelo, el archivo se concibe como una secuencia indexada de bloques de datos de tamaño homogéneo. Los programas pueden leer o escribir en cualquier bloque en un orden arbitrario, especificando el número de bloque relativo al inicio del archivo (`read_block(n)`), o desplazando libremente el puntero mediante llamadas como `lseek()` (Silberschatz et al., 2006, §10.2.2, p. 338). El acceso directo es imprescindible para sistemas de gestión de bases de datos relacionales (DBMS, *Database Management Systems*), almacenes clave-valor y motores de virtualización de discos rígidos, donde la aplicación requiere modificar registros dispersos sin leer exhaustivamente los datos precedentes. Asimismo, sustenta la técnica de proyección de archivos en memoria (*memory-mapped files* mediante la llamada `mmap()`), permitiendo mapear páginas de un archivo directamente en el espacio de direcciones virtuales del proceso para manipular su contenido como

si fuera un arreglo nativo en memoria RAM (Carretero Pérez et al., 2001, §8.2.4, pp. 427–428).

3. **Acceso indexado:** Diseñado para archivos de gran escala donde se requiere alta velocidad de localización basada en una clave de búsqueda (*key*) (Stallings, 2018, §12.2, pp. 558–560). El sistema mantiene un archivo de índice separado compuesto por punteros dirigidos hacia los bloques de datos del archivo principal. Para localizar un registro específico, el subsistema realiza una búsqueda binaria o recorre un árbol balanceado (árbol B o B+) sobre el índice, determinando la dirección física exacta del bloque de destino en una fracción mínima de tiempo en comparación con un barrido secuencial completo (McHoes & Flynn, 2011, §8, pp. 249–252). En sistemas que atienden volúmenes masivos de información, se construyen índices multinivel donde un índice primario direcciona a subíndices secundarios antes de alcanzar los bloques finales de datos (Silberschatz et al., 2006, §10.2.3, p. 339).

2. Taxonomía y catálogo de formatos de archivo

2.1. Mecanismos de identificación: extensiones, tipos MIME y números mágicos

La capacidad del sistema operativo para determinar la naturaleza y estructura interna de un archivo resulta decisiva para garantizar la correcta ejecución de los programas, evitar la corrupción de datos y prevenir vulnerabilidades de seguridad derivadas de la interpretación errónea de flujos binarios (Silberschatz et al., 2006, §10.1.3, p. 339). A lo largo de la evolución de las arquitecturas de software, se han consolidado tres mecanismos fundamentales para la identificación y enlace de formatos:

1. **Extensiones de nombre de archivo:** Introducidas de forma rígida en sistemas de disco como CP/M y MS-DOS bajo el formato de 8 caracteres de nombre y 3 de sufijo (convención 8.3), las extensiones constituyen sufijos alfanuméricos anexados tras el último carácter de punto en el nombre simbólico del archivo (Silberschatz et al., 2006, §10.1.3, p. 340). En sistemas Windows, el subsistema gráfico y el Registro central (`HKEY_CLASSES_ROOT`) asocian de manera determinista cada extensión con un controlador ejecutable específico (p. ej., asociando `.pdf` con Adobe Acrobat Reader). No obstante, esta aproximación presenta debilidades estructurales: la extensión es meramente nominal y puede ser alterada arbitrariamente por el usuario o por software malicioso (p. ej., técnicas de ofuscación mediante doble extensión como `documento.pdf.exe`), induciendo al usuario a ejecutar código arbitrario creyendo abrir un documento pasivo.
2. **Números mágicos y firmas binarias (*magic numbers*):** Como contraparte a la fragilidad de las extensiones, el estándar POSIX y los sistemas basados en UNIX implementan el análisis directo de los primeros bytes de la cabecera física del archivo (Carretero Pérez et al., 2001, §8.2.1, p. 422). Un número mágico es una secuencia estandarizada e invariable de bytes situada en un desplazamiento fijo (*offset*) dentro del archivo que identifica de forma irrefutable su especificación técnica, independientemente del nombre asignado. El comando estándar `file` y la biblioteca del sistema `libmagic` operan consultando bases de datos de patrones estructurados (`/etc/magic`), permitiendo reconocer ejecutables binarios ELF (`0x7F 0x45 0x4C 0x46`), imágenes PNG (`0x89 0x50 0x4E 0x47 0x0D 0x0A 0x1A 0x0A`) o contenedores ZIP (`0x50 0x4B 0x03 0x04`) incluso si el archivo carece de extensión o ha sido renombrado maliciosamente.

3. **Tipos de medios de Internet (MIME / IANA):** Estandarizados originalmente en la RFC 2046 por el IETF para el intercambio estructurado de correo electrónico multimedia (Freed & Borenstein, 1996), los tipos MIME representan una taxonomía universal en dos niveles (`tipo/subtipo`, como `text/plain`, `application/pdf` o `image/png`). En entornos de escritorio contemporáneos como GNOME y KDE bajo especificaciones de Freedesktop.org, el subsistema Shared MIME-Info combina heurísticas de extensión con inspección de números mágicos para despachar aplicaciones de forma unificada mediante descriptores `.desktop` y archivos de configuración `mimeapps.list`.

2.2. Catálogo técnico clasificado de cincuenta formatos de archivo

A continuación se presenta un catálogo técnico exhaustivo de cincuenta extensiones de archivo universales, agrupadas en siete dominios funcionales. Para cada formato se documenta su denominación estándar, tipo MIME oficial, propósito computacional, software de referencia para su generación y software de modificación o edición técnica especializada.

2.2.1. Documentos estructurados y ofimática

Esta categoría comprende formatos concebidos para la composición, maquetación, cálculo matricial y presentación visual de documentos textuales y tabulares, combinando texto codificado con metadatos de estilo, fuentes tipográficas y elementos multimedia incrustados. La Tabla 3 detalla los ocho formatos más representativos del sector.

Tabla 3: Formatos de documentos estructurados, ofimática e intercambio de información.

Extensión	Formato y tipo MIME	Software de creación	Software de modificación
`.txt`	Texto plano sin formato (ASCII / UTF-8). MIME: `text/plain`.	Vim, GNU nano, Bloc de notas de Windows.	GNU Emacs, Visual Studio Code, Sublime Text.
`.docx`	Microsoft Word Open XML (ECMA-376). MIME: `application/vnd.openxmlformats-....`	Microsoft Word, Google Docs (exportación).	LibreOffice Writer, WPS Office Writer, OnlyOffice.
`.odt`	Texto estándar OpenDocument (ISO/IEC 26300). MIME: `application/vnd.oasis.opendocument.text`.	LibreOffice Writer, Apache OpenOffice Writer.	Calligra Words, AbiWord, Collabora Online.
`.pdf`	Formato de Documento Portátil (ISO 32000). MIME: `application/pdf`.	pdfTeX, LuaLaTeX, Adobe Acrobat DC, LibreOffice.	Adobe Acrobat Pro, Master PDF Editor, pdfcpu, LibreOffice Draw.
`.rtf`	Texto enriquecido con control de formato. MIME: `application/rtf`.	WordPad de Windows, TextEdit de macOS.	LibreOffice Writer, Microsoft Word, Ted.
`.xlsx`	Hoja de cálculo Office Open XML (ECMA-376). MIME: `application/vnd.openxmlformats-....`	Microsoft Excel, Google Sheets (exportación).	LibreOffice Calc, Gnumeric, OnlyOffice Spreadsheet.
`.csv`	Valores separados por comas (RFC 4180). MIME: `text/csv`.	Intérpretes Python (`csv`), scripts AWK, LibreOffice Calc.	Microsoft Excel, Modern CSV, Vim, GNU nano.
`.pptx`	Presentación Office Open XML (ECMA-376). MIME: `application/vnd.openxmlformats-....`	Microsoft PowerPoint, Google Slides.	LibreOffice Impress, Apple Keynote, OnlyOffice.

2.2.2. Código fuente, scripts y lenguajes de programación

Los archivos de código fuente son flujos de texto plano codificados habitualmente en UTF-8 que contienen directivas gramaticales, declaraciones de estructuras y algoritmos destinados a ser procesados por compiladores, ensambladores o intérpretes. La Tabla 4 analiza nueve lenguajes de amplia difusión en sistemas y aplicaciones.

Tabla 4: Formatos de código fuente, scripts y lenguajes de programación.

Extensión	Formato y tipo MIME	Software de creación	Software de modificación
`.c`	Código fuente en C (ISO/IEC 9899). MIME: `text/x-c`.	Vim, Visual Studio Code, GNU nano.	CLion, GNU Emacs, Geany, Eclipse CDT.
`.cpp`	Código fuente en C++ (ISO/IEC 14882). MIME: `text/x-c++src`.	Neovim, Microsoft Visual Studio, Qt Creator.	CLion, Code::Blocks, Kate, Visual Studio Code.
`.py`	Scripts y módulos de Python. MIME: `text/x-python`.	Python IDLE, PyCharm, Visual Studio Code.	Spyder, Sublime Text, Vim, JupyterLab.
`.java`	Clases de código fuente Java. MIME: `text/x-java-source`.	IntelliJ IDEA, Eclipse IDE, Apache NetBeans.	Visual Studio Code, GNU Emacs, BlueJ.
`.sh`	Guion de intérprete POSIX / Bash. MIME: `application/x-sh`.	GNU Bash, GNU nano, Vim.	ShellCheck, Kate, Visual Studio Code, Gedit.
`.js`	Script ECMAScript / JavaScript (ECMA-262). MIME: `text/javascript`.	WebStorm, Visual Studio Code, Atom.	Sublime Text, Neovim, Brackets, Zed.
`.html`	Marcado de hipertexto HTML5 (W3C / WHATWG). MIME: `text/html`.	Adobe Dreamweaver, Visual Studio Code.	Sublime Text, Bluefish, GNU Emacs, Notepad++.
`.css`	Hojas de estilo en cascada CSS3 (W3C). MIME: `text/css`.	Visual Studio Code, WebStorm, precompiladores Sass.	Stylizer, Sublime Text, Vim, Espresso.
`.rs`	Código fuente en lenguaje Rust. MIME: `text/x-rust`.	Gestor Cargo (`cargo new`), Visual Studio Code.	RustRover, Helix, Neovim, CLion.

2.2.3. Gráficos digitales y mapas de bits

El almacenamiento de información visual se divide en dos paradigmas fundamentales: la rasterización matricial de píxeles (mapas de bits), dependiente de la resolución espacial y sujeta a compresión con o sin pérdida; y el modelado vectorial analítico, donde los objetos gráficos se describen como ecuaciones geométricas escalables. La Tabla 5 reúne ocho formatos estándar de la industria gráfica.

Tabla 5: Formatos de gráficos digitales, imágenes rasterizadas y gráficos vectoriales.

Extensión	Formato y tipo MIME	Software de creación	Software de modificación
`.png`	Gráficos de Red Portátiles (ISO/IEC 15948). MIME: `image/png`.	GIMP, Adobe Photoshop, Flameshot, Spectacle.	Krita, Paint.NET, Inkscape (exportador).
`.jpg`	Imagen comprimida JPEG (ISO/IEC 10918-1). MIME: `image/jpeg`.	Sensores de cámaras digitales, Adobe Lightroom.	Adobe Photoshop, Darktable, RawTherapee, GIMP.
`.svg`	Gráfico vectorial escalable basado en XML. MIME: `image/svg+xml`.	Inkscape, Adobe Illustrator, generadores TikZ.	CorelDRAW, Boxy SVG, Visual Studio Code.
`.gif`	Formato de intercambio de gráficos (GIF89a). MIME: `image/gif`.	Adobe Animate, conversores FFmpeg, GIMP.	ScreenToGif, Photoshop, Ezgif.
`.bmp`	Mapa de bits independiente de dispositivo. MIME: `image/bmp`.	Microsoft Paint, escáneres de digitalización.	ImageMagick, Paint.NET, GIMP, IrfanView.
`.webp`	Formato de compresión avanzada de Google. MIME: `image/webp`.	Utilidad `cwebp`, navegadores Chromium, Blender.	GIMP (módulo WebP), Adobe Photoshop, Krita.
`.tiff`	Formato de imagen etiquetada de alta fidelidad. MIME: `image/tiff`.	Escáneres industriales, microscopía, Photoshop.	Adobe Lightroom, ImageJ, GIMP.
`.psd`	Documento por capas nativo de Photoshop. MIME: `image/vnd.adobe.photoshop`.	Adobe Photoshop.	Krita, GIMP, Photopea.

2.2.4. Audio, vídeo y medios interactivos

Los formatos multimedia encapsulan flujos continuos de datos temporales sincronizados. Operan mediante esquemas de compresión psicoacústica o psicovisual para reducir el ancho de banda requerido por el almacenamiento masivo, o mediante códecs sin pérdidas que preservan íntegramente la señal de muestreo original. La Tabla 6 agrupa ocho formatos de audio y vídeo.

Tabla 6: Formatos de audio, vídeo y contenedores multimedia.

Extensión	Formato y tipo MIME	Software de creación	Software de modificación
`.mp3`	Audio comprimido MPEG-1 Layer III. MIME: `audio/mpeg`.	Audacity, codificador LAME, Adobe Audition.	REAPER, Sound Forge, FFmpeg.
`.flac`	Códec de audio libre sin pérdidas (Xiph.Org). MIME: `audio/flac`.	Codificador oficial `flac`, Exact Audio Copy.	Audacity, Ardour, foobar2000.
`.wav`	Flujo de modulación por impulsos codificados (PCM). MIME: `audio/wav`.	Grabadoras digitales, Pro Tools, Audacity.	Adobe Audition, Sound Forge Pro, FL Studio.
`.aac`	Codificación de audio avanzada (MPEG-4 Part 3). MIME: `audio/aac`.	Apple Logic Pro, Fraunhofer FDK AAC, FFmpeg.	Adobe Audition, Nero Recode, Audacity.
`.ogg`	Contenedor multimedia abierto (Vorbis/Opus). MIME: `audio/ogg`.	Herramientas `oggenc` / `opusenc`, Audacity.	REAPER, Ardour, Ocenaudio.
`.mp4`	Contenedor audiovisual MPEG-4 Part 14. MIME: `video/mp4`.	OBS Studio, cámaras de vídeo, Adobe Premiere Pro.	DaVinci Resolve, Kdenlive, HandBrake.
`.mkv`	Contenedor multimedia extensible Matroska. MIME: `video/x-matroska`.	MKVToolNix (`mkvmerge`), OBS Studio, FFmpeg.	HandBrake, Avidemux, Shotcut.
`.avi`	Intercalado de audio y vídeo de Microsoft. MIME: `video/x-msvideo`.	Cámaras DV heredadas, VirtualDub, Adobe Premiere.	VirtualDub2, Avidemux, FFmpeg.

2.2.5. Archivos comprimidos, empaquetados e imágenes de disco

El empaquetado agrupa múltiples archivos y directorios dentro de un flujo contiguo preservando sus metadatos jerárquicos, mientras que los algoritmos de compresión eliminan la redundancia estadística mediante esquemas como Huffman, Lempel-Ziv o LZMA. La Tabla 7 describe siete formatos universales de almacenamiento y distribución.

Tabla 7: Formatos de empaquetado, compresión de datos e imágenes de disco.

Extensión	Formato y tipo MIME	Software de creación	Software de modificación
`.zip`	Archivo comprimido PKZIP (ISO 21320-1). MIME: `application/zip`.	Utilidad `zip` de POSIX, 7-Zip, WinZip.	WinRAR, PeaZip, Ark de KDE.
`.tar`	Empaquetador de cinta POSIX (IEEE 1003.1). MIME: `application/x-tar`.	Utilidad `tar` de GNU / BSD.	7-Zip, Xarchiver, Ark.
`.gz`	Flujo comprimido con DEFLATE (RFC 1952). MIME: `application/gzip`.	Utilidad `gzip` de GNU.	7-Zip, PeaZip, WinRAR.
`.7z`	Contenedor de alta compresión LZMA/LZMA2. MIME: `application/x-7z-compressed`.	7-Zip (`7za`), p7zip.	PeaZip, WinRAR, Ark.
`.rar`	Archivo propietario Roshal Archive. MIME: `application/vnd.rar`.	WinRAR, RAR para línea de comandos.	WinRAR (edición directa de archivos RAR).
`.iso`	Imagen de disco óptico ISO 9660 / UDF. MIME: `application/x-iso9660-image`.	`genisoimage`, `mkisofs`, Brasero, ImgBurn.	UltraISO, PowerISO, ISOMaster.
`.deb`	Paquete binario de software para Debian. MIME: `application/vnd.debian.binary-package`.	`dpkg-deb`, `debuild`, herramientas Debian.	`dpkg-deb -R` (extracción y reempaquetado), Alien.

2.2.6. Binarios ejecutables, bibliotecas y volcados de memoria

Los archivos binarios del sistema contienen código máquina compilado directamente ejecutable por la unidad central de procesamiento (CPU), tablas de símbolos para el enlace dinámico y segmentos de memoria estructurados según la arquitectura del procesador y la interfaz binaria de aplicación (ABI, *Application Binary Interface*). La Tabla 8 presenta cinco formatos críticos del núcleo y espacio de usuario.

Tabla 8: Formatos binarios ejecutables, bibliotecas dinámicas y volcados del sistema.

Extensión	Formato y tipo MIME	Software de creación	Software de modificación
`.elf`	Formato Ejecutable y Enlazable (System V ABI). MIME: `application/x-executable`.	Enlazadores GNU `ld`, LLVM `lld`, compiladores GCC/Clang.	`objcopy`, `strip`, depuradores GDB, Ghidra.
`.exe`	Ejecutable portable de Windows (PE32/PE32+). MIME: `application/vnd.microsoft.portable-executable`.	Enlazador MSVC (`link.exe`), MinGW-w64 GCC.	Resource Hacker, CFF Explorer, x64dbg.
`.so`	Biblioteca de objetos compartidos en Linux/UNIX. MIME: `application/x-sharedlib`.	Compiladores GCC / Clang (`-shared -fPIC`).	`patchelf`, `strip`, `objcopy`.
`.dll`	Biblioteca de enlace dinámico de Windows. MIME: `application/x-msdownload`.	Microsoft Visual Studio, MinGW GCC.	DLL Export Viewer, Dependency Walker, IDA Pro.
`.core`	Volcado de memoria del proceso ante fallo crítico. MIME: `application/x-coredump`.	Núcleo del sistema operativo (manejador de `SIGSEGV`).	Depurador GNU (GDB), LLDB, editores hexadecimales.

2.2.7. Bases de datos, configuración y serialización estructurada

La persistencia de registros estructurados y la parametrización de servicios del sistema operativo requieren esquemas de serialización universales que permitan la deserialización rápida en memoria o la ejecución transaccional sobre esquemas relacionales. La Tabla 9 resume cinco formatos medulares de persistencia y serialización.

Tabla 9: Formatos de bases de datos, configuración y serialización estructurada.

Extensión	Formato y tipo MIME	Software de creación	Software de modificación
`.sql`	Guion de sentencias estructuradas ANSI SQL. MIME: `application/sql`.	Utilidades `mysqldump`, `pg_dump`, DBeaver.	DataGrip, Visual Studio Code, pgAdmin.
`.sqlite`	Base de datos relacional compacta sin servidor. MIME: `application/vnd.sqlite3`.	Motor de base de datos SQLite (`sqlite3` CLI).	DB Browser for SQLite, SQLiteStudio, DBeaver.
`.json`	Notación de objetos JavaScript (RFC 8259). MIME: `application/json`.	Serializadores de software (`json.dump`), Postman.	Visual Studio Code, procesador `jq`, Sublime Text.
`.xml`	Lenguaje de marcado extensible (W3C XML). MIME: `application/xml`.	Generadores de esquemas XML, servicios empresariales SOAP.	XMLSpy, Oxygen XML Editor, Notepad++.
`.yaml`	Lenguaje de serialización legible por personas. MIME: `application/yaml`.	Editores de manifiestos Kubernetes, Ansible, Docker.	PyCharm, Neovim, Visual Studio Code.

3. Arquitectura y componentes de los sistemas de archivos

3.1. Definición formal y modelo arquitectónico por capas

Un sistema de archivos (*file system*) es la estructura lógica, algorítmica y de metadatos mediante la cual el sistema operativo organiza, persiste, localiza, recupera, valida y protege colecciones de información sobre dispositivos físicos de almacenamiento secundario y terciario (Stallings, 2018, §12.1, p. 551). Lejos de constituir un bloque de software monolítico, el subsistema de archivos se implementa mediante un diseño fuertemente desacoplado y estructurado en capas de abstracción jerárquicas, donde cada nivel resuelve un aspecto

específico del aislamiento entre la semántica de usuario y la física del medio magnético, óptico o semiconductor (Silberschatz et al., 2006, §11.1, p. 370).

La arquitectura de software estándar adoptada por los sistemas operativos modernos se descompone en cinco niveles funcionales sucesivos, ilustrados en la Img. 2:

1. **Nivel de programas de aplicación:** Consume primitivas de entrada/salida a través de bibliotecas estándar (p. ej., `stdio.h` en C) y llamadas al sistema del núcleo.
2. **Sistema lógico de archivos (*logical file system*):** Gestiona los metadatos globales del sistema de archivos, la jerarquía de directorios, la comprobación de privilegios de usuario y los bloques de control de archivo (FCB/inodos). Asigna identificadores numéricos y garantiza la seguridad de acceso antes de transferir la operación a los niveles inferiores (Silberschatz et al., 2006, §11.1, p. 371).
3. **Módulo de organización de archivos (*file-organization module*):** Conoce el mapeo lógico del archivo y el método de asignación utilizado (contiguo, enlazado o indexado). Traduce los desplazamientos de bloques lógicos relativos al inicio del archivo $(0, 1, 2, \dots)$ en identificadores de bloques lógicos de disco (LBA, Logical Block Addressing). Asimismo, gestiona el rastreo de bloques libres del dispositivo (Silberschatz et al., 2006, §11.1, p. 371).
4. **Sistema básico de archivos (*basic file system*):** Transforma comandos genéricos de transferencia de bloques en solicitudes de E/S orientadas a dispositivos. Administra la memoria intermedia de almacenamiento en caché (*buffer cache* o caché de páginas) en la memoria RAM del sistema para minimizar los accesos físicos a los discos (Stallings, 2018, §12.1, p. 555).
5. **Control de E/S y manejadores de dispositivo (*I/O control and device drivers*):** Comunica directamente con los registros de control, buses y canales del hardware (SATA, NVMe, SCSI, USB), transfiriendo bloques mediante acceso directo a memoria (DMA, *Direct Memory Access*) y gestionando las interrupciones del controlador (Silberschatz et al., 2006, §11.1, p. 371).

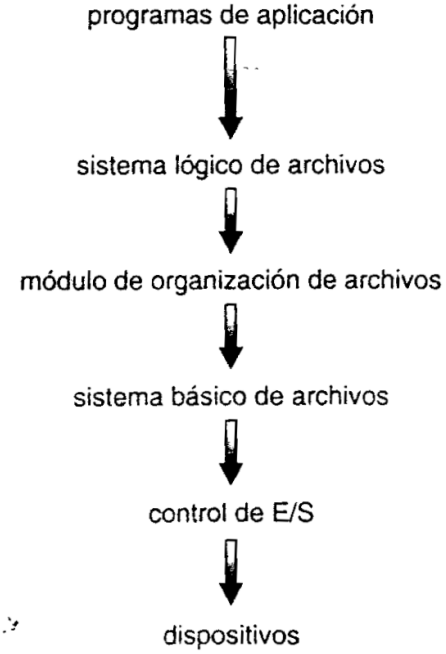


Imagen 2: Arquitectura por capas de software de un subsistema de archivos (Silberschatz et al., 2006, §11.1, p. 371).

3.2. Capa de abstracción: el Sistema de Archivos Virtual (VFS)

En un entorno computacional contemporáneo coexisten simultáneamente sistemas de archivos heterogéneos montados sobre una misma jerarquía de directorios: particiones raíz en ext4, volúmenes de respaldo en Btrfs, unidades externas en exFAT o NTFS, sistemas ópticos ISO 9660, recursos compartidos por red mediante NFS y pseudo-sistemas de archivos dinámicos del núcleo como `procfs` y `sysfs`. Para evitar que las aplicaciones deban programarse contra interfaces privativas de cada tecnología, el núcleo implementa una capa de indirección polimórfica denominada Sistema de Archivos Virtual (VFS, *Virtual File System*) (Stallings, 2018, §12.8, pp. 581–583).

El VFS define una interfaz orientada a objetos en C compuesta por cuatro estructuras canónicas fundamentales (Stallings, 2018, §12.8, pp. 583–585):

- **Objeto de superbloque (`struct super_block`):** Representa un sistema de archivos específico montado. Contiene metadatos del volumen (tamaño de bloque, banderas de solo lectura, identificador de dispositivo) y un puntero a su tabla de métodos polimórficos (`struct super_operations`), permitiendo al núcleo invocar operaciones de montaje, desmontaje y asignación de inodos de forma agnóstica.
- **Objeto inodo (`struct inode`):** Representa la abstracción de un archivo o directorio individual en memoria. Alberga sus atributos persistentes y un vector de operaciones (`struct inode_operations`) para manipular enlaces, permisos y nombres (`create`, `lookup`, `link`, `mkdir`).
- **Objeto entrada de directorio o *dentry* (`struct dentry`):** Modela un nodo en la ruta jerárquica, asociando una cadena alfanumérica con su respectivo inodo. El VFS mantiene estas estructuras en una memoria caché de alta velocidad denominada *dcache*, evitando exploraciones costosas en disco cada vez que un proceso resuelve rutas complejas como `/usr/bin/python3` (Stallings, 2018, §12.8, p. 584).
- **Objeto archivo (`struct file`):** Representa una instancia de apertura dinámica en memoria asociada a un descriptor de proceso. Mantiene el puntero de desplazamiento actual (offset), las banderas de apertura (`flags`) y la tabla de operaciones de lectura, escritura y posicionamiento (`struct file_operations`).

3.3. Estructuras en disco y en memoria principal

La implementación física de un sistema de archivos requiere la coexistencia de estructuras permanentes almacenadas en el medio no volátil y estructuras volátiles en memoria RAM para la aceleración operativa (Silberschatz et al., 2006, §11.2, pp. 372–374):

- **Estructuras en almacenamiento secundario:**
  - *Bloque de control de arranque (boot control block):* Ocupa los primeros sectores del volumen o partición (p. ej., el sector de arranque de partición VBR o el registro maestro de arranque MBR). Contiene las instrucciones máquina ejecutadas por la interfaz de firmware para cargar el núcleo del sistema operativo.
  - *Bloque de control de volumen (volume control block):* Denominado superbloque (*superblock*) en UNIX/Linux o tabla maestra de archivos (MFT) en NTFS. Almacena la geometría del volumen, recuento de bloques totales y libres, tamaño del bloque de asignación y punteros a los mapas de espacio libre (Silberschatz et al., 2006, §11.2, p. 373).
  - *Estructura de directorios física:* Bloques reservados que enlazan los nombres simbólicos legibles por humanos con sus respectivos identificadores numéricos

de inodo o punteros de bloque.
    - *Bloque de Control de Archivo (FCB):* Estructuras fijas (tabla de inodos en ext4) que albergan los atributos, marcas temporales y punteros a los bloques de datos de cada archivo (Carretero Pérez et al., 2001, §8.5.1, p. 461).
- **Estructuras dinámicas en memoria principal:**
    - *Tabla de montaje del sistema:* Lista enlazada de todos los sistemas de archivos montados, enlazando puntos de montaje con sus superbloques activos.
    - *Caché de inodos y entradas de directorio (icache / dcache):* Mantiene copias activas de los inodos recientemente consultados y de las ramas del árbol de directorios para evitar lecturas recurrentes de metadatos.
    - *Tabla global y tablas locales de archivos abiertos:* Coordinan los descriptores activos y punteros de desplazamiento entre procesos emparentados o independientes.

3.4. Métodos de asignación de bloques en almacenamiento secundario

El módulo de organización de archivos debe resolver el problema crítico de cómo asignar bloques físicos de almacenamiento a los archivos lógicos en crecimiento. La eficiencia del acceso secuencial, la latencia del acceso aleatorio y el grado de fragmentación dependen del método adoptado (Silberschatz et al., 2006, §11.4, pp. 380–388):

1. **Asignación contigua (*contiguous allocation*):** Cada archivo ocupa un conjunto de bloques consecutivos en el disco. Para acceder a cualquier bloque n del archivo, basta sumar n a la dirección del bloque inicial ($B_{dir} = B_{inicio} + n$), lo que otorga la máxima velocidad en acceso secuencial y directo, minimizando el desplazamiento de las cabezas lectoras en discos electromecánicos (Silberschatz et al., 2006, §11.4.1, p. 380). No obstante, genera fragmentación externa severa a medida que los archivos se borran y crean, requiriendo algoritmos de compactación periódicos. Asimismo, exige declarar o anticipar el tamaño final del archivo al momento de crearlo, produciendo fallos de asignación prematuros si el archivo supera el espacio continuo disponible (Stallings, 2018, §12.7, p. 570).
2. **Asignación enlazada (*linked allocation*):** Cada archivo se representa como una lista enlazada de bloques físicos dispersos. Cada bloque almacena una porción de datos y un puntero físico hacia el siguiente bloque de la cadena; la entrada de directorio registra únicamente las direcciones del primer y último bloque (Silberschatz et al., 2006, §11.4.2, p. 382). Elimina por completo la fragmentación externa y permite que un archivo crezca indefinidamente mientras existan bloques libres en el volumen. Sin embargo, resulta ineficiente para el acceso directo: localizar el bloque k requiere leer secuencialmente los $k-1$ bloques precedentes (complejidad $O(n)$). Además, presenta vulnerabilidad ante fallos de integridad: la pérdida o corrupción de un solo puntero provoca la pérdida irreparable del resto del archivo.
3. **Asignación basada en tabla de asignación (FAT):** Variante optimizada de la asignación enlazada adoptada en MS-DOS y Windows. Los punteros de encadenamiento se extraen de los bloques de datos y se agrupan en una tabla contigua al inicio del disco denominada Tabla de Asignación de Archivos (FAT, *File Allocation Table*) (Silberschatz et al., 2006, §11.4.2, p. 384). Al cargarse la tabla en memoria RAM, el acceso directo se acelera notablemente al recorrer la cadena de punteros en memoria principal sin

incurrir en lecturas físicas a disco.

4. **Asignación indexada (*indexed allocation*):** Resuelve las deficiencias del acceso directo reuniendo todos los punteros a los bloques de datos de un archivo en una estructura dedicada denominada bloque de índice (*index block*) (Silberschatz et al., 2006, §11.4.3, p. 385). El directorio almacena la dirección del bloque de índice, y el acceso a cualquier bloque lógico requiere una sola consulta directa a la tabla de índices, sin fragmentación externa. Para dar soporte a archivos de tamaños dispares, los sistemas implementan índices multinivel o esquemas combinados como el inodo tradicional de UNIX/Linux, el cual incorpora doce punteros directos, un puntero indirecto simple, un puntero indirecto doble y un puntero indirecto triple, posibilitando tanto el almacenamiento eficiente de archivos pequeños como el direccionamiento de archivos de escala multi-terabyte (Silberschatz et al., 2006, §11.4.3, pp. 386–387).

La Tabla 10 contrasta las propiedades operativas y estructurales de los métodos de asignación analizados.

Tabla 10: Comparativa técnica de métodos de asignación de espacio en almacenamiento secundario.

Método	Eficiencia de acceso	Sobrecarga de almacenamiento	Fragilidad y fragmentación
Contigua	Óptima tanto en acceso secuencial como en aleatorio ($O(1)$).	Mínima: solo requiere dirección de inicio y longitud en el FCB.	Sufre fragmentación externa grave; requiere compactación periódica.
Enlazada pura	Eficiente en acceso secuencial; muy deficiente en aleatorio ($O(n)$).	Pérdida de 4 u 8 bytes por bloque para alojar el puntero de enlace.	Nula fragmentación externa; muy vulnerable a corrupción de punteros.
Tabla FAT	Rápida en secuencial; moderada en aleatorio mediante recorrido en RAM.	Tabla fija proporcional al número total de bloques del volumen.	Nula fragmentación externa; cuello de botella por concurrencia en la FAT.
Indexada (Inodo)	Alta en secuencial y en aleatorio ($O(1)$ o $O(\log n)$ en multinivel).	Requiere bloques de índices dedicados incluso en archivos muy pequeños.	Nula fragmentación externa; gran tolerancia a fallos y alta escalabilidad.

3.5. Gestión del espacio libre en disco

Para asignar bloques con rapidez cuando un archivo se crea o expande, el sistema operativo debe rastrear de forma continua cuáles bloques físicos están disponibles en el soporte persistente. Las tres técnicas elementales son (Silberschatz et al., 2006, §11.5, pp. 388–391):

- **Vector o mapa de bits (*bit vector* / *bitmap*):** Cada bloque del disco se asocia a un bit individual (0 = libre, 1 = ocupado). Esta técnica es altamente compacta y permite que el núcleo localice el primer bloque libre o secuencias de bloques contiguos mediante instrucciones máquina especializadas de búsqueda de bits (`bsf` o `tzcnt` en arquitecturas x86_64) (Silberschatz et al., 2006, §11.5.1, p. 388). Su eficiencia máxima exige mantener el mapa íntegramente en memoria RAM.
- **Lista enlazada de bloques libres:** Conecta todos los bloques disponibles entre sí mediante punteros almacenados en los propios bloques libres, requiriendo únicamente mantener en el superbloque un puntero al primer bloque libre (Silberschatz et al., 2006,

§11.5.2, p. 389). No introduce sobrecarga de espacio adicional en disco, pero resulta lenta para asignaciones masivas contiguas.

- **Agrupación y recuento (*grouping and counting*):** Almacena las direcciones de n bloques libres en el primer bloque libre disponible, o registra pares estructurados formados por la dirección del primer bloque libre y el número de bloques continuos que le suceden (extents libres) (Silberschatz et al., 2006, §11.5.3, p. 390). Esta técnica es la base de los asignadores modernos en sistemas de archivos avanzados como ext4 y XFS, optimizando la asignación contigua a gran velocidad.

4. Análisis comparativo de sistemas de archivos

4.1. Criterios de evaluación y taxonomía arquitectónica

La evolución de los sistemas de archivos a lo largo de las últimas cuatro décadas refleja la constante adaptación del software de base frente al crecimiento exponencial en la capacidad de los medios persistentes, las demandas de concurrencia masiva en arquitecturas multiprocesador y la transición desde dispositivos electromecánicos hacia medios de estado sólido (Stallings, 2018, §12.7, pp. 575–578). Para evaluar y clasificar técnicamente un sistema de archivos, el análisis de ingeniería examina cuatro dimensiones estructurales fundamentales:

1. **Representación de metadatos y direccionamiento:** Define cómo el sistema mapea la identidad lógica del archivo sobre los bloques físicos del medio. Los esquemas abarcan desde tablas de asignación globales lineales en memoria (FAT32, exFAT), tablas maestras de registros relacionales extensibles (MFT en NTFS), árboles combinados de nodos de índice con extensiones contiguas (inodos con extents en ext4 y XFS), hasta árboles B balanceados polimórficos (Btrfs, APFS) y estructuras unificadas de almacenamiento en depósitos virtuales (*storage pools* en ZFS) (Silberschatz et al., 2006, §11.4, pp. 384–387).
2. **Límites máximos de direccionamiento:** Determinados por el ancho de palabra binaria asignado a los punteros de bloque y campos de longitud en los descriptores del sistema. Las implementaciones transitan desde los 32 bits de FAT32 (limitado a archivos de 4 GiB), pasando por los 48 o 64 bits habituales en ext4, NTFS, XFS y APFS (capaces de direccionar magnitudes en el orden de los tebibytes y exbibytes), hasta alcanzar los 128 bits teóricos de ZFS, cuya capacidad de direccionamiento supera la masa atómica de los medios físicos concebibles (OpenZFS Project, 2024).
3. **Mecanismos de consistencia y tolerancia a fallos:** Establecen la capacidad del sistema para preservar la integridad ante interrupciones abruptas de energía o caídas del núcleo. La taxonomía divide los sistemas en tres grupos: aquellos sin bitácora que exigen revisiones completas fuera de línea mediante escaneos de disco (FAT32, exFAT); sistemas con bitácora (journaling) que registran previamente transacciones de metadatos o datos completos para una recuperación acelerada en el arranque (NTFS, ext4, XFS, HFS+) (Silberschatz et al., 2006, §11.6, p. 392); y sistemas modernos con paradigma de copia en escritura (CoW, Copy-on-Write) e integridad criptográfica de extremo a extremo, los cuales garantizan consistencia intrínseca y autorreparación de datos frente a degradación física silenciosa (bit rot) mediante sumas de comprobación (checksums) en árbol Merkle (ZFS, Btrfs, APFS) (OpenZFS Project, 2024).

4. **Capacidades avanzadas de gestión de almacenamiento:** Funcionalidades integradas en el propio sistema de archivos que eliminan la necesidad de capas intermedias de software, tales como compresión transparente en tiempo real (LZ4, ZSTD, LZNT1), cifrado criptográfico granular nativo por archivo o volumen (fscrypt, EFS, FileVault), creación de instantáneas (snapshots) de solo lectura o lectura/escritura en tiempo constante $O(1)$, clonación de archivos sin duplicación física de bloques y particionamiento en subvolúmenes flexibles (Apple Inc., 2020; The Linux Kernel Organization, 2024a).

4.2. Tabla comparativa técnica de diez sistemas de archivos

La Tabla 11 sintetiza los parámetros operativos, arquitecturas de metadatos, límites de direccionamiento, esquemas de consistencia y ámbitos de despliegue preferente de diez sistemas de archivos de amplia difusión en la industria computacional.

4.3. Análisis de paradigmas de consistencia y almacenamiento

El diseño interno de un sistema de archivos define la estrategia adoptada para conciliar el rendimiento en operaciones de entrada/salida con la preservación incondicional de la coherencia de los datos ante fallos imprevistos de hardware o interrupciones de energía (Silberschatz et al., 2006, §11.6, pp. 392–394). En la práctica de la ingeniería computacional destacan tres grandes paradigmas:

1. **Paradigma de bitácora transaccional (journaling):** En los sistemas convencionales con actualización en el sitio (update-in-place) como ext4 o NTFS, modificar un archivo exige realizar múltiples escrituras independientes en sectores dispersos del disco (bloques de datos, mapas de bits, inodos y directorios). Si el suministro eléctrico se interrumpe antes de completar la secuencia, el volumen queda inconsistente y requiere escaneos prolongados con `fsck` o `chkdsk` (Stallings, 2018, §12.9, pp. 586–588). El journaling resuelve este problema reservando un área circular en disco denominada bitácora (journal). Antes de persistir cambios en las estructuras finales, el núcleo agrupa las operaciones en una transacción atómica y la registra de manera secuencial en la bitácora. Tras confirmarse la escritura física de la transacción (*commit*), el núcleo transfiere de forma asíncrona las modificaciones a sus ubicaciones finales en disco (checkpointing). En caso de caída, durante el arranque el núcleo examina únicamente la bitácora: descarta transacciones incompletas y reaplica las transacciones confirmadas pendientes en segundos (*replay*), garantizando consistencia inmediata (Silberschatz et al., 2006, §11.6, p. 393).
2. **Paradigma de copia en escritura (Copy-on-Write, CoW) y árboles Merkle:** Sistemas modernos como ZFS, Btrfs y APFS descartan la actualización en el sitio. Bajo CoW, los bloques de datos y metadatos existentes jamás se sobreescriben. Cuando un proceso modifica información, el asignador reserva un nuevo bloque físico libre donde graba la versión actualizada. La referencia en el nodo padre del árbol se actualiza en cascada de forma ascendente hasta alcanzar la raíz del árbol (*uberblock* o superbloque de anclaje) (OpenZFS Project, 2024). La confirmación de la transacción se reduce a una sola escritura atómica del puntero de raíz. Si ocurre una interrupción eléctrica, la raíz anterior sigue referenciando la versión coherente previa, prescindiendo de bitácoras de recuperación. Este esquema hace posible generar instantáneas (snapshots)

Tabla 11: Análisis comparativo exhaustivo de diez sistemas de archivos contemporáneos.

Sistema	Límites máx. (Archivo / Vol.)	Estructura de metadatos	Resiliencia y consistencia	Capacidades avanzadas	Ámbito de despliegue principal
FAT32	4 GiB / 2 TiB	Tabla de asignación de archivos de 32 bits (FAT); directorio plano de 32 bytes con soporte VFAT.	Sin bitácora; susceptible a inconsistencias tras apagones; requiere escaneo con FSCK o CHKDSK.	Compatibilidad universal multiplataforma; sin soporte de permisos nativos ni cuotas de usuario.	Memorias USB heredadas, tarjetas SD, partición de arranque UEFI (ESP).
exFAT	16 EiB / 128 PiB	Tabla FAT simplificada con mapa de bits de asignación contigua para optimizar archivos sin fragmentar.	Sin bitácora; soporte transaccional opcional en TexFAT; menor fragilidad que FAT32.	Nombres de archivo Unicode hasta 255 caracteres; sin compresión ni permisos POSIX nativos.	Tarjetas de memoria SDXC de alta capacidad y unidades externas de intercambio.
NTFS	16 TiB / 256 TiB	Tabla Maestra de Archivos (MFT) como base relacional; descriptores de archivo en registros de 1 KiB.	Bitácora transaccional para metadatos mediante `$LogFile`; recuperación atómica en fases.	Listas de control de acceso (ACL), flujos alternos (ADS), cifrado EFS, compresión y cuotas.	Partición de sistema y almacenamiento interno en Microsoft Windows NT/10/11.
ext4	16 TiB / 1 EiB	Inodos de 256 bytes por grupos de bloques; direccionamiento con árboles balanceados de extents.	Bitácora JBD2 con tres niveles (journal, ordered, writeback); checksums en metadatos.	Asignación diferida, preasignación contigua `fallocate()`, cifrado nativo fscrypt.	Sistema predeterminado en la mayoría de distribuciones GNU/Linux de escritorio y servidor.
Btrfs	16 EiB / 16 EiB	Estructura multirrámica basada en árboles B+ genéricos bajo copia en escritura (CoW).	Integridad de extremo a extremo con sumas CRC32C/SHA256 en datos y metadatos; autorreparación.	Instantáneas gravables $O(1)$, subvolúmenes dinámicos, compresión transparente zstd/LZO.	Partición raíz en Fedora y openSUSE; servidores NAS e infraestructuras contra corrupción.
XFS	8 EiB / 8 EiB	Grupos de asignación (AG) paralelos e independientes; inodos dinámicos y árboles B+ de extents.	Bitácora circular de metadatos de alto rendimiento; recuperación inmediata; checksums v5.	Escalabilidad extrema para E/S paralela masiva, asignación diferida y cuotas por proyecto.	Almacenamiento estándar en Red Hat Enterprise Linux (RHEL) y servidores de bases de datos.
ZFS	16 EiB / 256 ZiB	Fusión de sistema de archivos y administrador de volúmenes (zpools); árbol Merkle de 128 bits.	CoW transaccional estricto; suma de comprobación criptográfica por bloque; repara bit rot.	Instantáneas y clones instantáneos, compresión dinámica LZ4/ZSTD, desduplicación, RAID-Z.	Almacenamiento corporativo de misión crítica, servidores TrueNAS, FreeBSD e Illumos.
HFS+	8 EiB / 8 EiB	Archivo de catálogo jerárquico basado en árbol B*; bifurcaciones de recursos y datos separadas.	Bitácora de metadatos en diario; previene el escaneo exhaustivo del catálogo en arranque.	Codificación Unicode UTF-8 descompuesta, compresión nativa HFS, listas de control de acceso.	Sistema predeterminado en entornos Apple históricos (Mac OS 8.1 hasta macOS Sierra 10.12).
APFS	8 EiB / 8 EiB	Árboles B+ adaptados a memorias flash SSD; espacio compartido dinámico en un único contenedor.	Redirección en escritura para atomicidad transaccional sin bitácora; sumas en metadatos.	Clonación de archivos sin duplicación, instantáneas atómicas, cifrado nativo multi-clave.	Sistema nativo y obligatorio en todos los dispositivos Apple contemporáneos (macOS, iOS).
F2FS	3.93 TiB / 16 TiB	Estructura en registros segmentados (*Log-Structured File System*) optimizada para flash NAND.	Puntos de control periódicos (checkpointing) y registros de reversión (*roll-forward*).	Separación de datos calientes/fríos, recolección de basura adaptativa, soporte TRIM y compresión.	Teléfonos inteligentes Android, dispositivos embebidos y unidades de estado sólido de consumo.

atómicas en tiempo $O(1)$ congelando la raíz temporal. Con sumas de comprobación criptográficas incrustadas en los punteros padres de cada bloque, el sistema detecta y autorrepara la corrupción silenciosa de datos (bit rot) sobre arreglos redundantes (OpenZFS Project, 2024; The Linux Kernel Organization, 2024a).

3. **Paradigma estructurado en registros (*Log-Structured*) para memorias flash:** Las unidades de estado sólido basadas en tecnología flash NAND operan bajo restricciones físicas: leen y programan en páginas (de 4 a 16 KiB), pero borran únicamente en bloques de borrado completos (*erase blocks*, de 2 a 8 MiB) (Lee et al., 2015). La sobreescritura aleatoria tradicional fuerza al controlador a ejecutar ciclos de lectura-borrado-reescritura, disparando el factor de amplificación de escritura (WAF, *Write Amplification Factor*) y reduciendo la vida útil del silicio. Sistemas como F2FS transforman todas las escrituras aleatorias en un flujo estrictamente secuencial de datos en segmentos limpios, separando la información en flujos calientes, tibios y fríos. Mediante recolección de basura adaptativa en segundo plano y órdenes TRIM de SATA/NVMe, F2FS minimiza la degradación y optimiza el rendimiento sostenido del medio (Lee et al., 2015).

5. Sistemas de archivos en sistemas operativos contemporáneos

La selección del sistema de archivos en los sistemas operativos responde a criterios de compatibilidad, tolerancia a fallos, seguridad del núcleo y optimización para medios persistentes. A continuación se examina la arquitectura de almacenamiento en cuatro plataformas contemporáneas.

5.1. GNU/Linux Mint 21.2 «Victoria»

GNU/Linux Mint 21.2 («Victoria»), basado en Ubuntu 22.04.2 LTS con núcleo Linux 5.15 LTS, adopta **ext4** (*Fourth Extended Filesystem*) como sistema predeterminado en la raíz (`/`) (The Linux Mint Team, 2023). Emplea inodos de 256 bytes, árboles de extensiones (extents) y bitácora JBD2 en modo `data=ordered`, garantizando que los datos alcancen el medio persistente antes de asentar metadatos (The Linux Kernel Organization, 2024b). El intercambio opera mediante archivo dinámico (`/swapfile`) sobre ext4 vía `fibmap`, con el arranque UEFI en **FAT32** sobre la partición ESP (`/boot/efi`).

Mint 21.2 integra la herramienta de restauración **Timeshift** (The Linux Mint Team, 2023). Con ext4 opera en modo RSYNC mediante enlaces duros (*hard links*) para capturar instantáneas diferenciales sin duplicar archivos intactos. Alternativamente, soporta **Btrfs** con subvolúmenes `@` (raíz) y `@home` (usuarios); bajo este esquema, Timeshift conmuta al modo nativo BTRFS para capturar instantáneas CoW en tiempo constante $O(1)$, posibilitando la reversión inmediata del sistema desde GRUB ante actualizaciones fallidas (The Linux Mint Team, 2023).

5.2. Ubuntu 23.04 «Lunar Lobster»

Ubuntu 23.04 («Lunar Lobster»), con núcleo Linux 6.2 e instalador Subiquity/Flutter, mantiene **ext4** como sistema primario en escritorio y servidores (Canonical Ltd., 2023).

Opera con direccionamiento de 64 bits (`64bit`), asignación diferida (*delayed allocation*) y sumas CRC32C (`metadata_csum`), blindando inodos y mapas de bloques frente a corrupción silenciosa (The Linux Kernel Organization, 2024b). Conserva soporte para **OpenZFS** (*ZFS on root*) con depósitos (zpools) `rpool` y `bpool`, permitiendo a `zsys` capturar instantáneas previas a cada invocación de `apt` (Canonical Ltd., 2023; OpenZFS Project, 2024).

En Ubuntu 23.04, el sistema comprimido de solo lectura **Squashfs** distribuye aplicaciones en paquetes **Snap** vía `snapd` (Canonical Ltd., 2023; Lougher & The Linux Kernel Organization, 2024). Aplicaciones como Firefox no residen dispersas en `/usr`, sino en imágenes Squashfs inmutables (comprimidas con XZ o LZO) asociadas en el arranque a dispositivos de bucle invertido (`/dev/loop`) y montadas bajo `/snap/paquete/revisión`. Este confinamiento con AppArmor y espacios de nombres (*namespaces*) asegura aislamiento estricto, con el arranque UEFI en **FAT32** sobre ESP (Canonical Ltd., 2023).

5.3. Apple macOS 10.15 «Catalina»

macOS 10.15 («Catalina») adoptó **APFS** (*Apple File System*) sustituyendo a HFS+ en el almacenamiento interno (Apple Inc., 2020). Catalina redefinió la arquitectura mediante **volúmenes gemelos dedicados** en un contenedor compartido APFS (*shared container*): el volumen de sistema (`Macintosh HD`), montado de solo lectura en la raíz `/` bajo protección SIP (*System Integrity Protection*) con el núcleo XNU y marcos esenciales (`/System`); y el volumen de datos (`Macintosh HD - Data`), montado en `/System/Volumes/Data` para usuarios (`/Users`), aplicaciones (`/Applications`) y configuraciones (`/private`) (Apple Inc., 2020).

Para asegurar compatibilidad con POSIX y un espacio de nombres continuo, Apple implementó **enlaces firmes sintéticos (*firmlinks*)** (Apple Inc., 2020). Un *firmlink* es un túnel bidireccional en el núcleo y metadatos de APFS que enlaza de forma transparente directorios de solo lectura con sus pares en datos (proyectando `/Users` hacia `/System/Volumes/Data/Users`). La seguridad física del contenedor se complementa con **FileVault 2**, mediante claves AES-XTS independientes gestionadas por el coprocesador Apple T2 (Apple Inc., 2020).

5.4. Microsoft Windows 10

En Windows 10 (22H2), el sistema de archivos obligatorio para el sistema (`C:`) es **NTFS** (*New Technology File System*, versión 3.1), exigido por `winload.efi` con clústeres de 4 KiB (Microsoft Corporation, 2024b). NTFS articula la Tabla Maestra de Archivos (MFT, Master File Table) en registros de 1024 bytes (con atributos residentes en archivos pequeños), la bitácora `$LogFile` y el registro USN Journal (`$UsnJrnl`) para atomicidad transaccional, y el control de acceso por listas DACL/SACL con identificadores SID, cifrado nativo EFS y cifrado completo mediante **BitLocker** (Microsoft Corporation, 2024b; Stallings, 2018).

Para volúmenes secundarios, Windows 10 emplea una partición ESP en **FAT32** para UEFI, soporta **exFAT** en medios extraíbles e incorpora **ReFS** (*Resilient File System*) en Workstation y Enterprise para depósitos masivos (*Storage Spaces*) con CoW e integridad verificable, sin arranque ni compresión nativa (Microsoft Corporation, 2024a). La Tabla 12 resume los roles y subsistemas analizados.

Tabla 12: Sistemas de archivos implementados en sistemas operativos contemporáneos.

Sistema Operativo	Partición raíz	Partición datos	Tecnologías y subsistemas secundarios
GNU/Linux Mint 21.2	ext4 (predet.) / Btrfs	ext4 / Btrfs (`@home`)	Instantáneas Timeshift (RSYNC o CoW), `swapfile` en ext4, FAT32 en ESP.
Ubuntu 23.04	ext4 (predet.) / OpenZFS	ext4 / OpenZFS	Paquetes Snap sobre Squashfs inmutable (`/dev/loop`), `zsys`, FAT32 en ESP.
macOS 10.15 Catalina	APFS (*read-only*)	APFS (`Data`)	Túneles sintéticos firmlinks, cifrado FileVault 2 (Apple T2), SIP.
Microsoft Windows 10	NTFS obligatorio (v3.1)	NTFS	Bitácora `$LogFile`, registro USN, BitLocker, ReFS en Workstation, FAT32 en ESP.

6. Conclusiones personales

La investigación y el análisis estructural confirman que el sistema de archivos no constituye un organizador pasivo de datos, sino uno de los subsistemas de abstracción más complejos y críticos del sistema operativo. La transición desde registros rígidos hacia el flujo continuo no estructurado de bytes (stream of bytes), consolidado por UNIX y POSIX, redefinió la relación entre el núcleo y el espacio de usuario. Al delegar la interpretación en las aplicaciones y reservar al núcleo la asignación eficiente, protección de memoria y multiplexación de descriptores, se unificó el tratamiento de archivos ordinarios, tuberías y controladores. Asimismo, el Sistema de Archivos Virtual (VFS) demostró que el desacoplamiento polimórfico entre llamadas al sistema estandarizadas y estructuras físicas en disco es indispensable para garantizar la interoperabilidad en entornos heterogéneos con múltiples tecnologías coexistiendo en un mismo espacio jerárquico.

El análisis de paradigmas de consistencia confirma una transformación sustancial en el almacenamiento persistente. Frente a la actualización en el sitio (update-in-place) y la sobrecarga por doble escritura del journaling tradicional (`fsck` o `chkdsk`), las arquitecturas basadas en copia en escritura (CoW, Copy-on-Write) y árboles balanceados Merkle —materializadas en ZFS, Btrfs y APFS— establecen un estándar superior de resiliencia. Al registrar cada modificación como una nueva asignación física atómica propagada hacia la raíz del volumen, erradican los estados corruptos intermedios. Esta propiedad garantiza consistencia inmediata ante caídas abruptas y optimiza el respaldo al permitir instantáneas (snapshots) y clonación en tiempo constante $O(1)$ sin duplicar bloques.

La evolución del software de base permanece ligada a la física del hardware. La sustitución de discos rotatorios por memorias flash NAND derribó premisas algorítmicas de mitigación de retardos mecánicos. Las memorias de estado sólido imponen restricciones: la asimetría entre páginas de programación y bloques de borrado exige algoritmos que minimicen el factor de amplificación de escritura. Tecnologías como F2FS demuestran que el sistema de archivos debe dialogar con la capa FTL, sincronizando la recolección de basura y las órdenes TRIM para extender la durabilidad del silicio. Paralelamente, los sistemas operativos contemporáneos integran la seguridad e inmutabilidad en el núcleo: volúmenes gemelos con firmlinks en macOS Catalina, imágenes Squashfs inmutables en Snap bajo Ubuntu, y bitácoras con USN Journal en Windows 10. En conclusión, el sistema de archivos trasciende la intermediación de bloques para afianzarse como una plataforma integral de integridad, cifrado y resiliencia determinista.

Referencias

Apple Inc. (2020). *Apple File System Guide*. Consultado el 2 de octubre de 2026, desde https://developer.apple.com/documentation/foundation/file_system/about_apple_file_system

Canonical Ltd. (2023). *Ubuntu 23.04 (Lunar Lobster) Release Notes*. Consultado el 2 de octubre de 2026, desde https://discourse.ubuntu.com/t/lunar-lobster-release-notes/34885

Carretero Pérez, J., García Carballeira, F., Miguel Anasagasti, P. d., & Pérez Costoya, F. (2001). *Sistemas operativos: una visión aplicada* (1.ª ed.). McGraw-Hill/Interamericana de España.

Deitel, H. M., Deitel, P. J., & Choffnes, D. R. (2004). *Operating systems* (3.ª ed.). Pearson Prentice Hall.

Freed, N., & Borenstein, N. (1996). *Multipurpose Internet Mail Extensions (MIME) Part Two: Media Types* (inf. téc. N.º RFC 2046). IETF. Consultado el 2 de octubre de 2026, desde https://www.rfc-editor.org/rfc/rfc2046

Lee, C., Sim, D., Hwang, J.-Y., & Cho, S. (2015). F2FS: A New File System for Flash Storage. *USENIX ;login: 40*(4), 6-14. Consultado el 2 de octubre de 2026, desde https://www.usenix.org/conference/fast15/technical-sessions/presentation/lee

Lougher, P., & The Linux Kernel Organization. (2024). *Squashfs 4.0 - A squashed read-only filesystem for Linux*. Consultado el 2 de octubre de 2026, desde https://docs.kernel.org/filesystems/squashfs.html

McHoes, A. M., & Flynn, I. M. (2011). *Sistemas operativos* (6.ª ed.). Cengage Learning Editores.

Microsoft Corporation. (2024a). *Resilient File System (ReFS) overview*. Consultado el 2 de octubre de 2026, desde https://learn.microsoft.com/es-es/windows-server/storage/file-systems/refs-overview

Microsoft Corporation. (2024b). *Visión general de NTFS*. Consultado el 2 de octubre de 2026, desde https://learn.microsoft.com/es-es/windows-server/storage/file-systems/ntfs-overview

OpenZFS Project. (2024). *OpenZFS Documentation and Architecture*. Consultado el 2 de octubre de 2026, desde https://openzfs.github.io/openzfs-docs/

Silberschatz, A., Galvin, P. B., & Gagne, G. (2006). *Fundamentos de sistemas operativos* (7.ª ed.). McGraw-Hill/Interamericana de España.

Stallings, W. (2018). *Operating systems: Internals and design principles* (9.ª ed.) [Global Edition]. Pearson Education Limited.

The Linux Kernel Organization. (2024a). *Btrfs Documentation*. Consultado el 2 de octubre de 2026, desde https://docs.kernel.org/filesystems/btrfs.html

The Linux Kernel Organization. (2024b). *ext4 Data Structures and Algorithms*. Consultado el 2 de octubre de 2026, desde https://docs.kernel.org/filesystems/ext4/index.html

The Linux Mint Team. (2023). *Linux Mint 21.2 «Victoria» User Guide and Release Documentation*. Consultado el 2 de octubre de 2026, desde https://www.linuxmint.com/rel_victoria_whatsnew.php